- *Use the respective embeddings generator.* Use the appropriate embeddings constructor corresponding to the specific model you are working with. For example, when generating embeddings for the OpenAI model, utilize the `OpenAIEmbeddings` constructor.

6.2.4 Full example of preparing and storing documents with LangChain

This expanded example builds upon the previous retrieval-based chain by introducing an intermediate step to convert user questions into standalone questions. This addition aims to improve the accuracy of retrieving the answer.

Files: retrieval.openai.js, retrieval.google.js

The code for this example project is in a single file in the ch06 folder. You can use that example by running this command in the source folder if using Google:

```
$ node retrieval.google.js
```

If you are using Open AI, there is a separate file named retrieval.google.js in the same folder:

```
$ node retrieval.openai.js
```

You'll need an OpenAI API key to access their REST API or a Google API key for Google services. The appendix provides instructions for setting up a secret key for OpenAI and other providers:

```
// .env

OPENAI_API_KEY=<SECRET_KEY>
GEMINI_API_KEY=<YOUR_GEMINI_API_KEY>
```

Let's call out the key differences and enhancements in this code. First, it introduces a `standaloneQuestionChain` that transforms the user's input into a concise standalone question. This step helps to remove ambiguity and context-dependent elements from the original query. Then the overall chain structure is more complex. It first generates a standalone question. Then it uses this standalone question for retrieval. Finally, it combines the original question and retrieved context and generates an answer. The retriever is now part of a more sophisticated chain (`retrieverChain`) that takes the standalone question as input, retrieves relevant documents, and formats them as a string. The example also shows multiple queries, with two invocations of the chain: one with a question about AI (which should be answerable from the context) and another that is out of scope.

Feel free to use additional queries to test the performance of the chain and to explore how well it retrieves information based on different types of questions. You can